

How Input Size Affects the Speed of Algorithms:

Selection Sort and Merge Sort

The two most important factors when deciding an algorithm to use within a computer program are speed and computation. The number of computations done by an algorithm can be expressed using its 'Time Complexity'. This is calculated using a general analysis of the algorithm, favouring the dominating formulae as they are most important for large input sizes. There are three main time complexities that are considered when exploring algorithm efficiency, these are:

Big-O - The time complexity of the algorithm in its worst case scenario, its asymptotic upper bound.

Big-Omega - The time complexity of the algorithm in its best case scenario, its asymptotic lower bound.

Big-Theta - The combined complexity of both Big-O and Big-Omega, defined as both the upper and lower asymptotic bounds of the algorithm's time complexity.

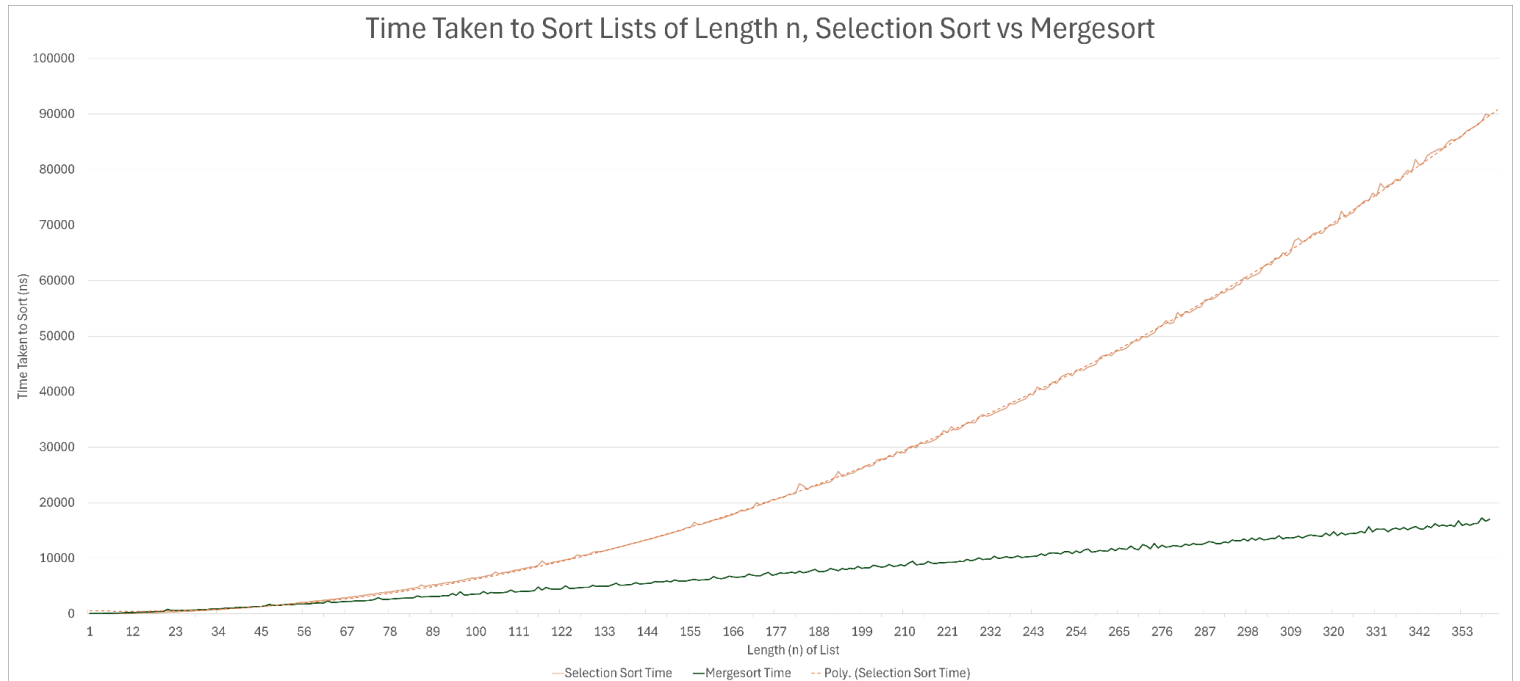
The time complexity can then be used to compare algorithm efficiency fairly, as it depends on the actual computational cost. However, time complexity, while having time in the name, cannot be used to accurately determine the time taken for the algorithm to complete its task.

To show this, we can implement two different sorting algorithms and compare the time taken for them to be completed. Selection Sort has a time complexity of $O(n^2)$, while considered one of the slower sorting algorithms, it is much simpler to implement than other algorithms. Mergesort has a time complexity of $O(n \log n)$, which is only slightly more efficient than Selection Sort at low input sizes, but is significantly better with large inputs. Mergesort is considerably more difficult to implement than Selection Sort because of its use of recursion, however the trade-off - especially in systems that know they will have large inputs - can be worth the effort.

To collect the results of this comparison, I ran a randomised test for both algorithms. The test repeats 360 times, one for each length of a list (n) that I deemed necessary to sufficiently display the data gathered in a graph. For each list length (n), a list is randomly generated, each entry in the list being a random number between 0 and 250 in order to ensure that a good deal of random numbers are chosen whilst still having some repeats, as this simulates an average case for both algorithms. The system time (in nanoseconds) is recorded prior to calling the sorting algorithm and once it is done, the system time is then recorded again, the difference between the two then gives us the time that the algorithm took to complete the sort, which is saved for later. This is repeated 4000 times for a single value of n before calculating the average time taken for that list length. Overall the test for one of the two algorithms is conducted over 1,440,000 generated lists, it is then repeated for the other algorithm, reaching a total of 2,880,000 generated lists that were sorted by

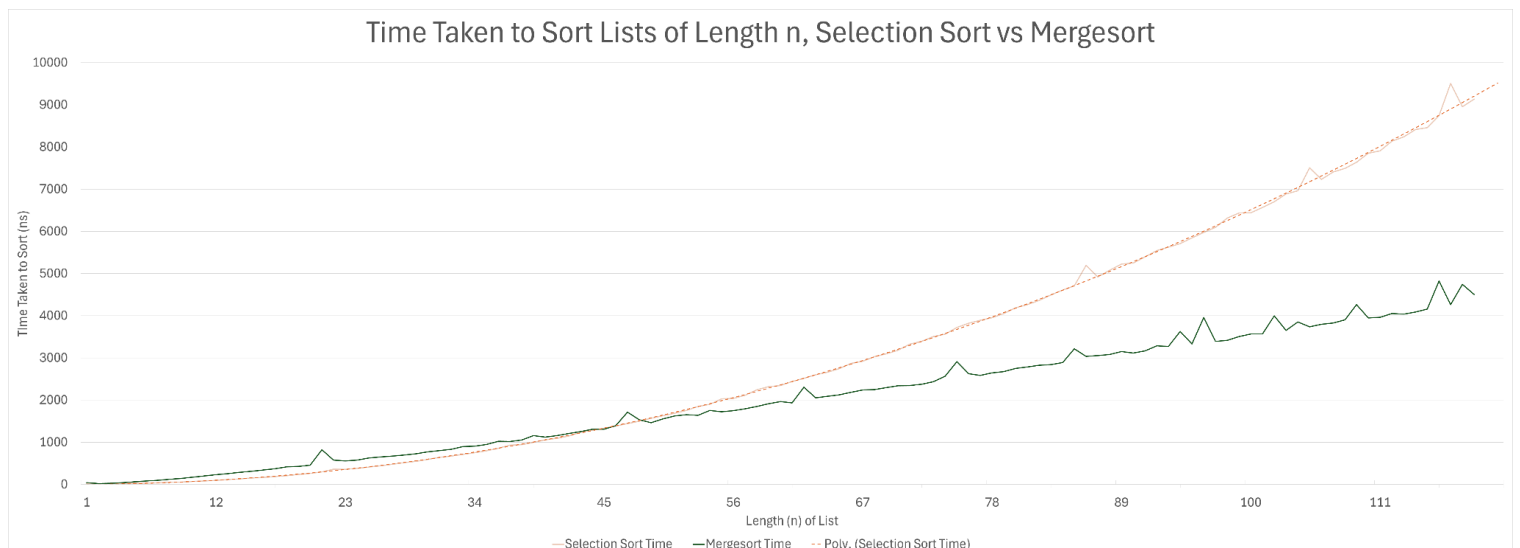
both algorithms to get an accurate set of data. The data is then written into a .csv file so that it can be used to plot a line graph that shows the average time taken to complete a sort for each list length from 0 to 360. Since the list generation is automated, and the size of the tests easily changeable, these tests are very flexible and repeatable.

The result is this graph:



Which follows our understanding of the efficiency of the two algorithms.

The time taken by both of them follow a similar trend, where at low inputs Selection Sort is faster than Mergesort, but as the input sizes increase, Mergesort overtakes Selection Sort and becomes much faster. We can see that the crossover takes place at $n = 45$, which, considering the amount of data that is used by businesses and researchers, is a relatively low input size for Selection Sort to be overtaken.



The overall trends for the time taken do not match that of their algorithmic time complexity.

An issue I ran into when conducting these tests was that the first few lengths of n would be discordantly larger than they should be when considering the rest of the data. This issue was caused by the start-up time of the Java virtual machine used when running the whole program, and the time taken to retrieve the instructions from the call to the Selection Sort and Mergesort functions that I created. To get around this, I granted my Java installation and IntelliJ 'Realtime' priority in task manager, and repeated the first 10 list lengths for Selection Sort, and repeated the entire test for mergesort, so that the virtual machine could setup spaces in my computer's cache to store the data needed to properly test the algorithms. This resulted in the first results more closely following what was expected and reduced the variance in results that were gathered from mergesort due the cpu having already set up enough memory space to store the recursive data.

In conclusion, the speed of algorithms has been shown to be dependent on input size, however not in the same way that time complexity is. I found that the speed of Selection Sort was quickly overtaken by the speed of Mergesort, at around 12.5% of the way through the tests.